三、数据结构部分（赵友先老师）

3.1 数组和链表的优缺点比较

	数组	链表
优点	1）在内存中，数组是一块连续的内存； 2）数组内的数据随机访问性比较强； 3）数组的查找速度块。	1）插入删除速度块； 2）内存利用率高，不会浪费内存； 3）大小不固定，进行动态拓展比较灵活。
缺点	1）插入和删除效率较低； 2）数组大小固定，不能进行动态的拓展； 3）可能会浪费内存。	1）不能随机查找，只能依次访问，查询效率低

3.2 如何判断环型链表(已修改)

```
bool isLoop(ListNode *head)
{
    if(head == NULL || head->next == NULL)
     { //参数检查
        return false;
    }

    ListNode *slow = head->next;
    ListNode *fast = head->next->next;

    while(fast != NULL && fast->next != NULL)
    {
        if(slow == fast)
        {          //判断成环
            return true;
        }
        slow = slow->next;          //slow 走一步
        fast = fast->next->next;    //fast 走两步
    }
    return false;
}
```

华清远见武汉中心嵌入式学科面试宝典

3.3 描述一种你熟悉的排序算法及其时间复杂度

算法名称	程序框架（数据规模为 n）	时间复杂度	空间复杂度	是否稳定

冒泡排序	for(i=0; i<n-1; i++)
	{
	for(j=0; j<n-i-1; j++)
	{
	if(a[j] > a[j+1])
	{
	int temp = a[j];
	a[j] = a[j+1];
	a[j+1] = temp;
	}
	}
	}
		$O(n^2)$	O(1)	稳定

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典

快速排序	int Quick_sort_partition(int a[], int low, int high)
	{
	int pivot = a[low];
	while(low < high)
	{
	while(a[high] >= pivot && low < high)
	high--;
	a[low] = a[high];
	while(a[low] <= pivot && low < high)
	low++;
	a[high] = a[low];
	}
	a[low] = pivot;
	return low;
	}
	
	void Quick_sort(int a[], int low, int high)
	{
	if(low < high)
	{
	int pivot = Quick_sort_partition(a, low, high);//第一次划分
	Quick_sort(a,low,pivot - 1);
	Quick_sort(a,pivot + 1,high);
	}
	//Quick_sort_show(a);
	}
		O(nlog$_2$n)
		最坏情况下时间复杂度：O(n^2)
			在最坏情况下，递归深度可以达到 n（数组的长度），这将导致 O(n) 的空间复杂度。然而，在平均情况下，快速排序的空间复杂度是 O(log n)，这是因为平均递归深度是 log n。	不稳定

3.5 内存碎片是什么意思，该如何去避免

内存碎片即“碎片的内存”描述一个系统中所有不可用的空闲内存。这些碎片内存之所以不能被使用是因为他们小且不连续，负责动态分配内存的分配算法无法再将它们分配给新的变量。内存碎片的成因或

华清远见武汉中心嵌入式学科面试宝典

大或小取决于内存管理算法及程序的编程细节。此外，内存碎片分为内部碎片和外部碎片：

内部碎片的产生：在页式虚拟存储系统中，用户作业的地址空间被划分成若干大小相等的页面，存储空间也分成也页大小相等的物理块，但一般情况下，作业的大小不可能都是物理块大小的整数倍，因此作业的最后一页中仍有部分空间被浪费掉了，换句话说，页面末尾浪费的内存空间就是内部碎片。

外部碎片的产生： 假设当某个客户请求一个 43 字节的内存块时，因为所有的内存分配必须起始于可被 4、8 或 16 整除，所以它可能会获得 44 或 48 字节等稍大一点的字节，而这多分配的几个字节就是外部内存碎片。最常见的就是结构体字节对齐导致结构体实际空间比成员空间之和要大；

在不能配置内存分配算法的情况下，用户有几种可行的策略来减少内存碎片：

1）尽可能的避免反复分配、释放同一类型的变量，可修改“局部申请内存+局部释放内存”的方式为“全局变量”，或者增加标志位指明变量空间中的数据是否可用；

2）设计结构体成员时，尽可能地考虑字节对齐问题。将同类型的结构成员放在一起，将占用字节数较多的成员放置在结构体末尾；

3）事先申请一大段内存，之后的变量按照自己的使用情况安置到已开辟的空间中，避免反复申请；不过这种操作比较复杂，除非有必要，否则不应在应用层实现；

4）禁止动态内存分配能较少内存碎片带来的负面影响（禁止函数 malloc()及关键字 new 的使用），常见于安全性要求较高的场合；

5）使用内存池机制来管理；

3.6 链表的逆序

采用双指针法：

```
/*
 * 反转链表
 * 参数：
 *   head：指向链表头节点的指针
 * 返回值：
 *    data_t 类型，表示操作的结果，ERROR 表示失败，OK 表示成功
 * 功能：
 *    将给定的链表进行反转，即链表中的节点顺序反转
 *    如果链表为空或仅有一个节点，则不进行操作直接返回 OK
 *    否则，通过一系列操作将链表节点的指针方向反转，实现链表的反转
 */
data_t Linklist_reverse(Linklist head)
{
```

华清远见武汉中心嵌入式学科面试宝典

```
    // 检查链表是否为空或只有一个节点，如果是，则直接返回 OK
    if(head == NULL || is_empty(head))
        return ERROR;
    // 初始化指针 p 和 q，分别指向头节点的下一个节点和下下个节点
    Linklist p = NULL,q = NULL;
    p = head->next;
    q = head->next->next;
    // 遍历链表，直到 q 指向最后一个节点的下一个节点（即 q 为 NULL）
    while(q != NULL)
    {
        // 保存当前节点 p 的下一个节点，以便后续操作
        p->next = q->next;
        // 将节点 q 的 next 指针指向前一个节点 p，实现指针反转
        q->next = head->next;
        // 将节点 q 插入到链表的头部，即 head 的 next 位置
        head->next = q;
        // 移动 q 指针到 p 的下一个节点，继续下一轮循环
        q = p->next;
    }
    // 链表反转完成后，返回 OK 表示操作成功
    return OK;
}
```

3.7 链表的排序

```
/*
 * 函数名：Linklist_sort
 * 功能：对链表进行排序
 * 参数：Linklist head - 链表的头指针
 * 返回值：data_t - 排序成功返回 OK，失败返回 ERROR
```

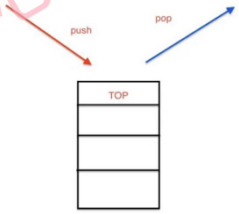
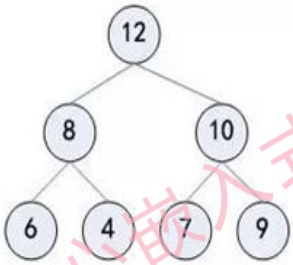
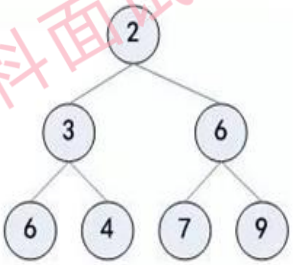
华清远见武汉中心嵌入式学科面试宝典

```
 * 说明：该函数使用插入排序算法对链表进行升序排序。它遍历链表，
 * 将每个节点插入到已排序的部分中合适的位置。
 */
data_t Linklist_sort(Linklist head)
{
   // 如果链表为空，直接返回错误
   if(head == NULL)
       return ERROR;
   // 初始化指针 p, q, r 用于链表操作
    Linklist p = NULL, q = NULL, r = NULL;
    p = head->next;
    head->next = NULL;
   // 遍历链表，对每个节点进行插入排序
    while(p != NULL)
    {
       q = p->next;
       r = head;

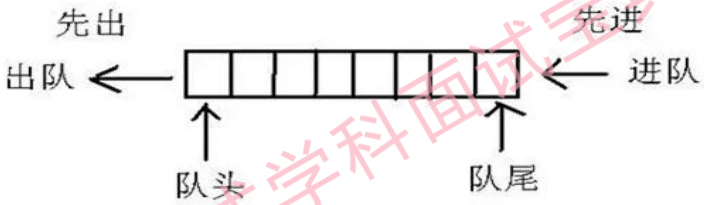
     // 找到当前节点 p 在已排序部分的插入位置
       while(r->next != NULL && r->next->data < p->data)
           r = r->next;
     // 将节点 p 插入到合适的位置
       p->next = r->next;
       r->next = p;
       p = q; // 继续下一个节点的排序
    }
    return OK; // 排序成功，返回 OK
}
```

华清远见武汉中心嵌入式学科面试宝典

3.8 队列、堆、栈的区别

栈	栈 是一种遵循 **后进先出（LIFO）** 原则的有序集合。按照常识理解就是有序的挤公交，**最后上车**的人会在门口，然后门口的人会**最先下车.** **栈是限定仅在表尾进行插入和删除操作的线性表。**我们把允许插入和删除的一端称为**栈顶**，另一端称为**栈底**，不含任何数据元素的栈称为空栈。栈的特殊之处在于它限制了这个线性表的插入和删除位置，它始终只在栈顶进行  **栈的应用——递归**
堆	**堆是一种经过排序的树形数据结构，每个节点都有一个值，通常我们所说的堆的数据结构是指**二叉树。所以堆在数据结构中通常可以被看做是一棵树的数组对象。而且堆需要满足一下两个性质： （1）堆中某个节点的值总是不大于或不小于其父节点的值； （2）堆总是一棵完全二叉树。 **堆分为两种情况，有最大堆和最小堆。**将根节点最大的堆叫做最大堆或大根堆，根节点最小的堆叫做最小堆或小根堆. 大根堆  小根堆 
队列	**队列是只允许在一端进行插入操作、而在另一端进行删除操作的线性表。** 允许插入的一端称为**队尾**，允许删除的一端称为**队头**。它是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作，

华清远见武汉中心嵌入式学科面试宝典

	和栈一样，队列是一种操作受限制的线性表。  队列是一种先进先出的数据结构，又称为先进先出的线性表，简称 FIFO（First In First Out）结构.
区别	• **操作原则：**栈是 LIFO，队列是 FIFO，堆则是基于元素的优先级（最大或最小）。 • **数据访问：**栈只能在一端进行数据操作，队列在两端操作（一端入队，一端出队），堆则是在树结构的根节点进行操作。 • **数据结构：**栈可以是数组或链表实现，队列通常用链表或数组实现（需要两个指针来管理头部和尾部），堆通常用数组实现，以保持元素的有序性。 • **用途：**栈常用于处理递归和回溯问题，队列用于任务调度和缓冲，堆用于高效的数据排序和优先级管理。

3.9 合并两个有序链表

```
/**
 * 合并两个有序链表
 * 该函数接收两个有序链表 l1 和 l2，并将它们合并为一个有序链表返回
 * @param l1 第一个有序链表的头指针
 * @param l2 第二个有序链表的头指针
 * @return 合并后的有序链表的头指针
 */
struct ListNode* mergeTwoLists(struct ListNode* l1, struct ListNode* l2)
{
    // 如果 l1 为空，直接返回 l2，反之亦然
    // 这是递归的终止条件，当其中一个链表遍历完时，直接返回另一个链表
    if(l1 == NULL) return l2;
    if(l2 == NULL) return l1;
    // 比较两个链表当前节点的值
```

华清远见武汉中心嵌入式学科面试宝典

```
        // 如果 l1 当前节点的值小于 l2 当前节点的值，那么 l1 当前节点应该放在前面
    if(l1->val < l2->val){
            // 递归调用 mergeTwoLists 函数，将 l1 的下一个节点和 l2 作为参数传入
            // 这样就可以继续比较并合并后面的节点
            l1->next = mergeTwoLists(l1->next,l2);
            // 返回 l1，继续向上一层递归
            return l1;
        }
        // 否则，l2 当前节点放在前面
        }
     else{
            // 递归调用 mergeTwoLists 函数，将 l1 和 l2 的下一个节点作为参数传入
            // 这样就可以继续比较并合并后面的节点
            l2->next = mergeTwoLists(l1,l2->next);
            // 返回 l2，继续向上一层递归
            return l2;
    }
```

3.10. 二分法查找代码编写

函数原型 int serach(int *nums, int numslen, int target);

```
/**
 * @param nums int 整型一维数组
 * @param numsLen int nums 数组长度
 * @param target int 整型
 * @return int 整型
 * 在排序数组中使用二分查找算法搜索目标值。
```

华清远见武汉中心嵌入式学科面试宝典

```
 * @param nums 指向已排序整数数组的指针。
 * @param numsLen 数组的长度。
 * @param target 要搜索的目标值。
 * @return 目标值在数组中的索引，如果未找到则返回-1。
 */
int search(int* nums, int numsLen, int target )
{
    // 如果数组为空，直接返回-1，表示未找到目标值。
    if(numsLen==0)
        return -1;
    // 初始化二分查找的起始、中间和结束索引。
    int low=0;
    int mid=numsLen/2;
    int high=numsLen-1;
    // 当起始索引小于等于结束索引时，继续进行二分查找。
 while(low<=high)
    {
        // 如果中间索引的值等于目标值，返回中间索引作为找到的位置。
        if(nums[mid]==target)
            return mid;
        // 如果中间索引的值小于目标值，说明目标值在右半部分，调整起始索引。
        else if(nums[mid]<target)
{
            low=mid+1;
            mid=(high+low)/2;
        }
        // 如果中间索引的值大于目标值，说明目标值在左半部分，调整结束索引。
        else
        {
            high=mid-1;
```

华清远见武汉中心嵌入式学科面试宝典

```
            mid=(high+low)/2;
        }
    }

    // 如果循环结束还没有找到目标值，返回-1。
    return -1;
}
```

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典